

C++ Programming: Problem Solving and Programming

Chapter 9 Recursion

Objectives

- What is the use of recursion
- Understand how recursion works in functions
- When recursion can be used
- What is recursive algorithm
- Understand some practical examples of using recursion

Recursion / Iteration ?

- There are usually two ways to solve a particular problem:
 - Iteration (looping)
 - Recursion
- Which method is better?
- In addition to the nature of the problem, the other key factor in determining the best solution method is efficiency

Recursion / Iteration ?

- An iterative solution is more obvious and easier to understand than a recursive solution
- Iteration is faster and less memory consuming
- If the nature of a problem is inherently recursive, easier to choose recursion, e.g. factorials, fibonacci numbers, divide & conquer algorithms like sorting, searching, etc.
- Coding for recursion is shorter, simpler and easier to understand

Introduction to Recursion

- It is a powerful technique that solves a problem by solving a smaller problem of the same type.
- A recursive solution normally reduces a problem to one or more smaller problems of the same kind until a ***base case (stopping case)*** is reached.
- Recursive function can call itself with different parameters. Based on this feature, recursion can be used as an alternative to iteration (looping).

Rules of Recursion

a) Base case / Stopping case

There must be one or more base cases which can be solved without recursion. The value produced by the call function without resorting to recursion is usually the stopping case value.

b) General case

To solve complex problem recursively, the call function must make progress toward a base case.

c) Finite

The problem is solvable and can be reduced to base cases.

Recursion Algorithm

General Algorithm:

```
// base case  
if the base case is reached then  
    solve it  
else  
    // general case  
    redefine the problem using recursion
```

Recursion Algorithm

- Define a problem in terms of a smaller version of itself
 - Every recursive function must have one (or more) base cases
 - recursive function is a function that calls itself
 - The general case must eventually reduce to the base case
 - The base case stops the recursion

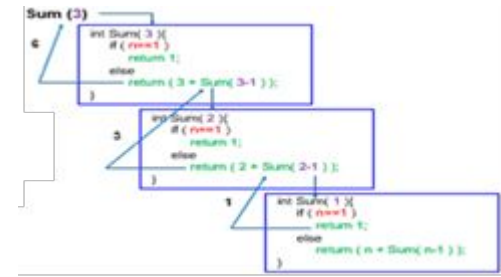
Recursion Functions

- Think of a recursive function as having infinitely many copies of itself
- Every call to a recursive function has
 - Its own code
 - Its own set of parameters and local variables
- After completing a particular recursive call
 - Control goes back to the calling function, which is the previous call
- The previous call begins from the point immediately following the recursive call

Recursion Functions - Design

- Understand the problem requirements
- Determine the limiting conditions by identifying the base case(s) and providing solutions to each of the base case(s)
- Identify the general case(s) and provide a solution to each general case in terms of the smaller version of itself

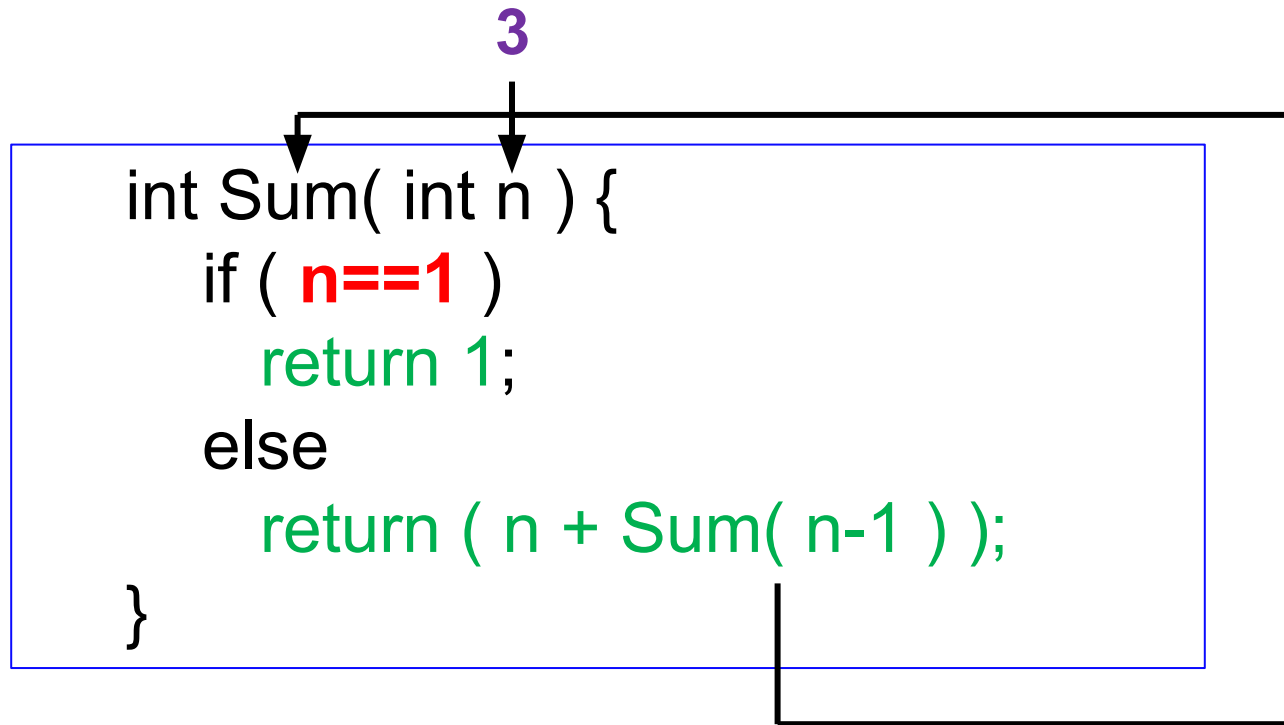
Recursion – Activation Frames



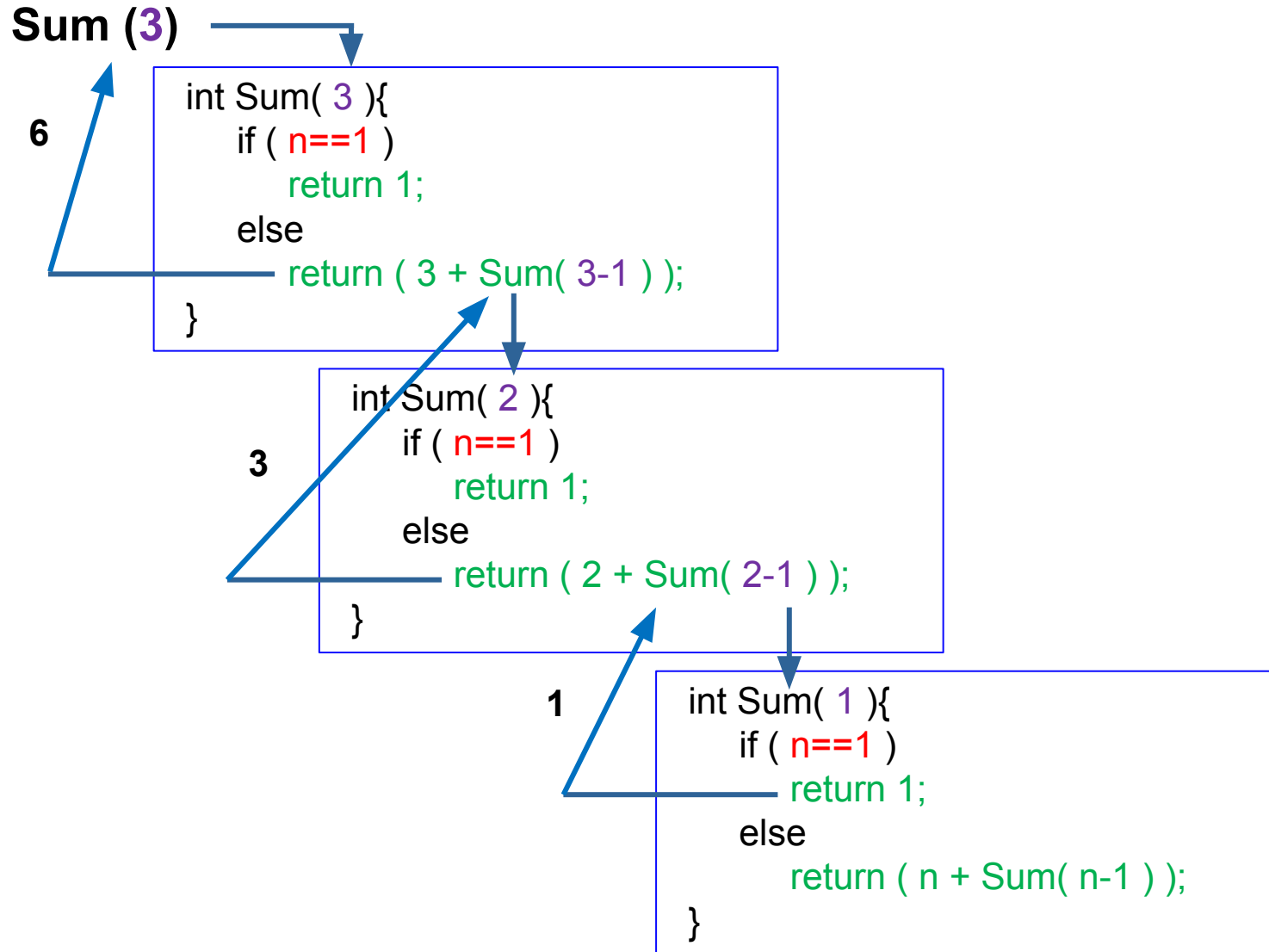
- The block of memory created on the whenever a function is invoked.
- It stores all the information needed to manage that specific function call until it finishes execution.
- It contains:
 - **Return address** – where the program should continue after the function finishes.
 - **Parameters/arguments** – the values passed to the function.
 - **Local variables** – variables declared inside the function.
 - **Saved state** – such as the previous stack pointer, registers, etc., so execution can resume correctly.

Recursion Implementation

a) Sum up numbers from 1 to N

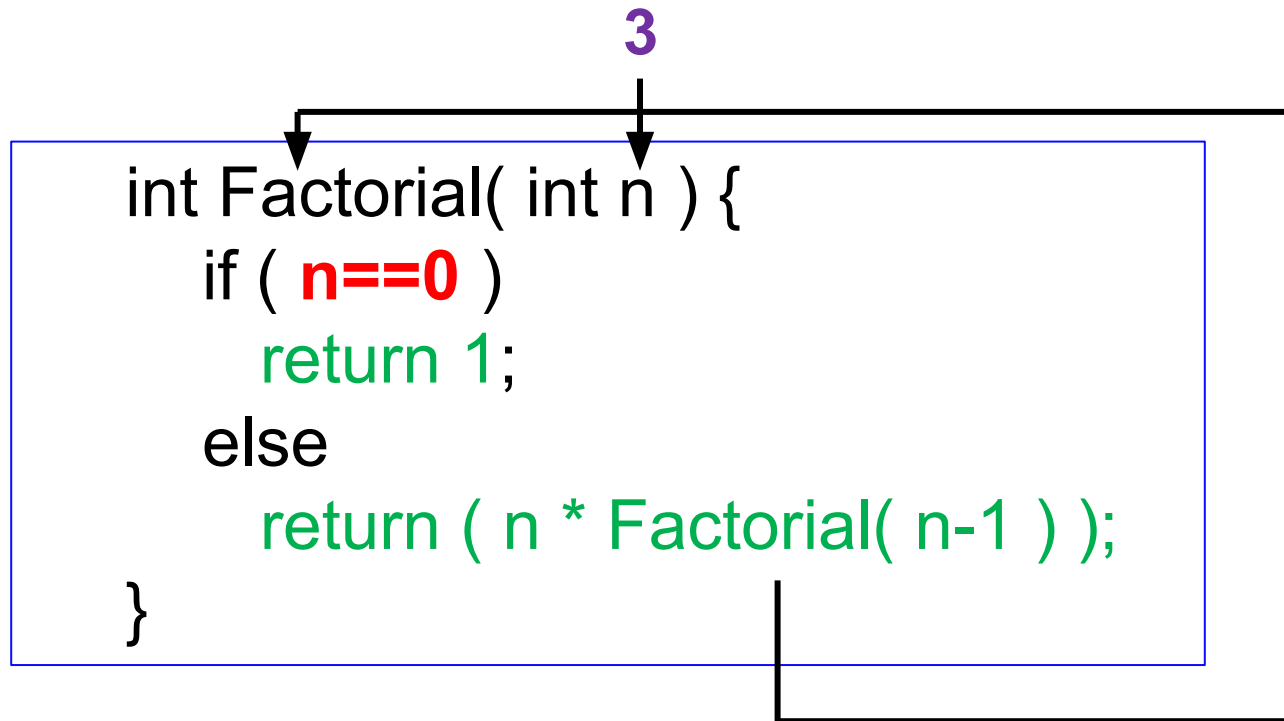


Recursion Implementation



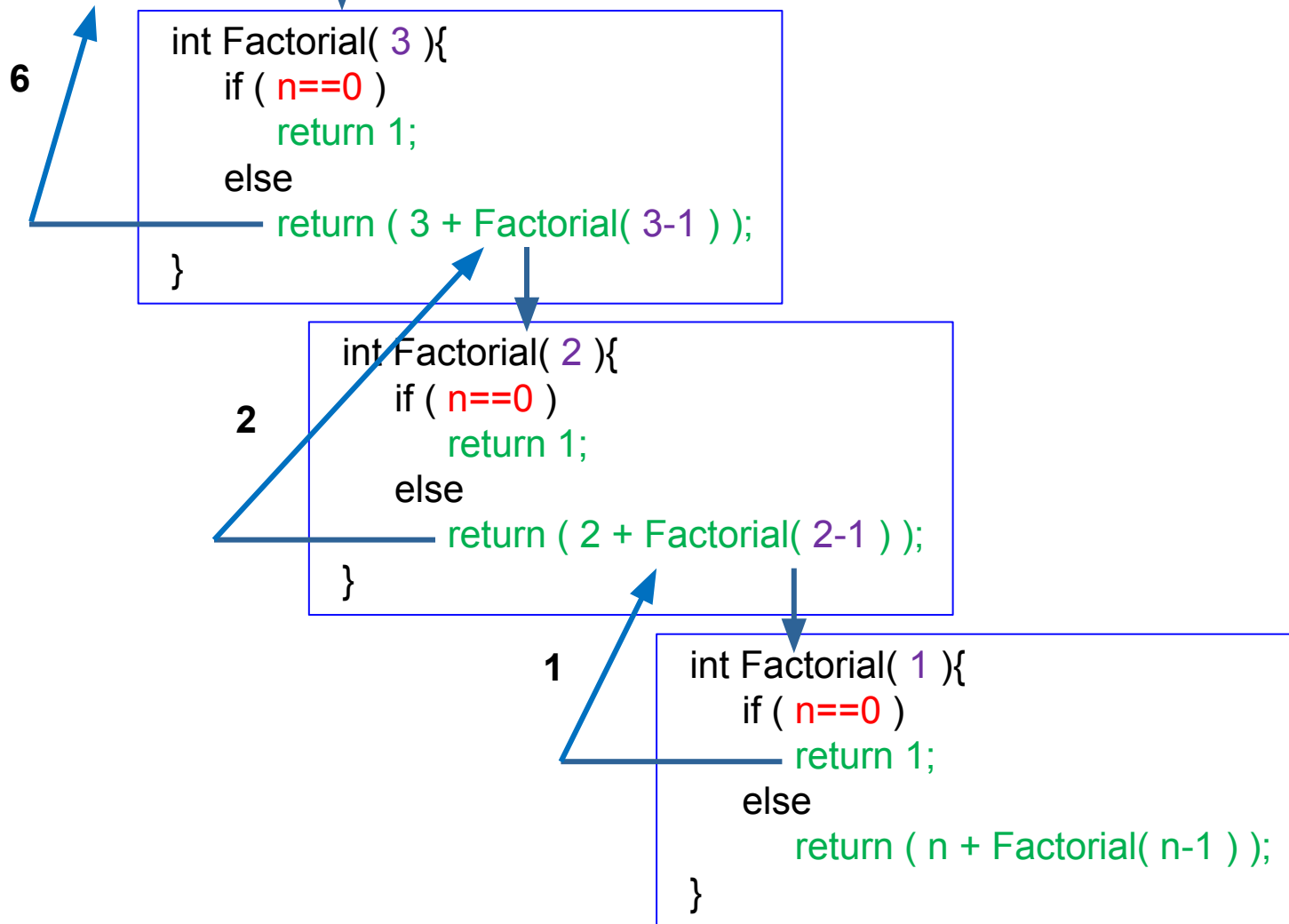
Recursion Implementation

b) Factorial: product of integral values from 1 to N



Recursion Implementation

Factorial (3)



Recursion Implementation

c) Fibonacci: It is a series of numbers in which each number is the sum of the previous two numbers.

```
int Fibonacci( int n ) {  
    if ( n<=0 )  
        return 0;  
    else if ( n==1 )  
        return 1;  
    else  
        return (Fibonacci (n -1) + Fibonacci( n - 2 ));  
}
```

Recursion Implementation

Result:
Fibonacci(3) = 2

Fibonacci (3)

```
if ( n <= 0 ) false  
else if ( n == 1 ) false  
else return ( Fibo(2) + Fibo(1));
```

Fibonacci (2)

```
if ( n <= 0 ) false  
else if ( n == 1 ) false  
else return ( Fibo(1) + Fibo(0));
```

Fibonacci (1)

```
if ( n <= 0 ) false  
else if ( n == 1 ) true  
return 1
```

Fibonacci (1)

```
if ( n <= 0 ) false  
else if ( n == 1 ) true  
return 1
```

Fibonacci (0)

```
if ( n <= 0 ) true  
return 0
```

Recursion - Summary

- Recursion is more elegant in the codes (simpler and cleaner), natural, but can be more memory-heavy.
- Iteraction is more efficient, faster, less memory consumed, but sometimes less readable.